



09. PyTorch Model Deployment Exercise Solutions

Welcome to the 09. PyTorch Model Deployment exercise solutions.

Your objective is to write code to satisfy each of the exercises below.

Some starter code has been provided to make sure you have all the resources you need.

Note: There may be more than one solution to each of the exercises.

Resources

1. These exercises/solutions are based on [section 09. PyTorch Model Deployment](#) of the Learn PyTorch for Deep Learning course by Zero to Mastery.
2. See a live [walkthrough of the solutions \(errors and all\) on YouTube](#) (but try the exercises yourself first!).
3. See [all solutions on the course GitHub](#).

Note: The first section of this notebook is dedicated to getting various helper functions and datasets used for the exercises. The exercises start at the heading "Exercise 1: ...".

Get various imports and helper functions

The code in the following cells prepares imports and data for the exercises below. They are taken from [09. PyTorch Model Deployment](#).

```
# For this notebook to run with updated APIs, we need torch 1.12+ and torchvision 0.13+
try:
    import torch
    import torchvision
    assert int(torch.__version__.split(".")[1]) >= 12, "torch version should be 1.12+"
    assert int(torchvision.__version__.split(".")[1]) >= 13, "torchvision version should be 0.13+"
    print(f"torch version: {torch.__version__}")
    print(f"torchvision version: {torchvision.__version__}")
except:
    print(f"[INFO] torch/torchvision versions not as required, installing nightly versions.")
    !pip3 install -U torch torchvision torchaudio --extra-index-url https://download.pytorch.org/whl/cu113
    import torch
    import torchvision
    print(f"torch version: {torch.__version__}")
    print(f"torchvision version: {torchvision.__version__}")
```

```
[INFO] torch/torchvision versions not as required, installing nightly versions.
Looking in indexes: https://pypi.org/simple, https://download.pytorch.org/whl/cu113
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.9.1)
Requirement already satisfied: torchvision in /usr/local/lib/python3.12/dist-packages (0.24.1)
Requirement already satisfied: torchaudio in /usr/local/lib/python3.12/dist-packages (2.9.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.1)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.3.20 in /usr/local/lib/python3.12/dist-packages (from torch) (3.3.20)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
```

```
Requirement already satisfied: triton==3.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.5.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from torchvision) (2.0.2)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.12/dist-packages (from torchvision) (10.4.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch) (3.0.3)
torch version: 2.9.1+cu128
torchvision version: 0.24.1+cu128
```

```
# Continue with regular imports
import matplotlib.pyplot as plt
import torch
import torchvision

from torch import nn
from torchvision import transforms

# Try to get torchinfo, install it if it doesn't work
try:
    from torchinfo import summary
except:
    print("[INFO] Couldn't find torchinfo... installing it.")
    !pip install -q torchinfo
    from torchinfo import summary

# Try to import the going_modular directory, download it from GitHub if it doesn't work
try:
    from going_modular.going_modular import data_setup, engine
    from helper_functions import download_data, set_seeds, plot_loss_curves
except:
    # Get the going_modular scripts
    print("[INFO] Couldn't find going_modular or helper_functions scripts... downloading them from GitHub.")
    !git clone https://github.com/mrdbourke/pytorch-deep-learning
    !mv pytorch-deep-learning/going_modular .
    !mv pytorch-deep-learning/helper_functions.py . # get the helper_functions.py script
    !rm -rf pytorch-deep-learning
    from going_modular.going_modular import data_setup, engine
    from helper_functions import download_data, set_seeds, plot_loss_curves
```

```
device = "cuda" if torch.cuda.is_available() else "cpu"
device
```

```
'cuda'
```

Get data

Want to download the data we've been using in PyTorch Model Deployment:

https://www.learnpytorch.io/09_pytorch_model_deployment/#1-getting-data

```
# Download pizza, steak, sushi images from GitHub
image_path = download_data(source="https://github.com/mrdbourke/pytorch-deep-learning/raw/main/data/pizza_steak_sushi",
                           destination="pizza_steak_sushi_20_percent")

image_path

[INFO] data/pizza_steak_sushi_20_percent directory exists, skipping download.
PosixPath('data/pizza_steak_sushi_20_percent')
```

```
# Setup directory paths to train and test images
train_dir = image_path / "train"
test_dir = image_path / "test"
```

Preprocess data

Turn images into tensors using same code as PyTorch Paper Replicating section 2.1 and 2.2:

https://www.learnpytorch.io/08_pytorch_paper_replicating/#21-prepare-transforms-for-images

```
# Create image size (from Table 3 in the ViT paper)
IMG_SIZE = 224

# Create transform pipeline manually
manual_transforms = transforms.Compose([
```

```

    transforms.Resize((IMG_SIZE, IMG_SIZE)),
    transforms.ToTensor(),
])
print(f"Manually created transforms: {manual_transforms}")

```

```

Manually created transforms: Compose(
  Resize(size=(224, 224), interpolation=bilinear, max_size=None, antialias=True)
  ToTensor()
)

```

```

# Set the batch size
BATCH_SIZE = 32 # this is lower than the ViT paper but it's because we're starting small

```

```

# Create data loaders
train_dataloader, test_dataloader, class_names = data_setup.create_data_loaders(
    train_dir=train_dir,
    test_dir=test_dir,
    transform=manual_transforms, # use manually created transforms
    batch_size=BATCH_SIZE
)

```

```

train_dataloader, test_dataloader, class_names

```

```

(<torch.utils.data.dataloader.DataLoader at 0x7c8880db7ad0>,
 <torch.utils.data.dataloader.DataLoader at 0x7c8880d4aed0>,
 ['pizza', 'steak', 'sushi'])

```

```

# Get a batch of images
image_batch, label_batch = next(iter(train_dataloader))

```

```

# Get a single image from the batch
image, label = image_batch[0], label_batch[0]

```

```

# View the batch shapes
image.shape, label

```

```

(torch.Size([3, 224, 224]), tensor(2))

```

```

# Plot image with matplotlib
plt.imshow(image.permute(1, 2, 0)) # rearrange image dimensions to suit matplotlib [color_channels, height, width] -
plt.title(class_names[label])
plt.axis(False);

```

sushi



Exercise 1. Make and time predictions with both feature extractor models on the test dataset using the GPU (`device="cuda"`).

- Compare the model's prediction times on GPU vs CPU - does this close the gap between them? As in, does making predictions on the GPU make the ViT feature extractor prediction times closer to the EffNetB2 feature extractor prediction times?
- You'll find code to do these steps in [section 5. Making predictions with our trained models and timing them](#) and [section 6. Comparing model results, prediction times and size](#).

Train two models on Pizza, Steak, Sushi data

Need:

- Trained EffNetB2 feature extractor
- Trained ViT feature extractor

✓ EffNetB2

See function creation source here: https://www.learnpytorch.io/09_pytorch_model_deployment/#31-creating-a-function-to-make-an-effnetb2-feature-extractor

```
def create_effnetb2_model(num_classes:int=3,
                        seed:int=42):
    """Creates an EfficientNetB2 feature extractor model and transforms.

    Args:
        num_classes (int, optional): number of classes in the classifier head.
            Defaults to 3.
        seed (int, optional): random seed value. Defaults to 42.

    Returns:
        model (torch.nn.Module): EffNetB2 feature extractor model.
        transforms (torchvision.transforms): EffNetB2 image transforms.
    """
    # 1, 2, 3. Create EffNetB2 pretrained weights, transforms and model
    weights = torchvision.models.EfficientNet_B2_Weights.DEFAULT
    transforms = weights.transforms()
    model = torchvision.models.efficientnet_b2(weights=weights)

    # 4. Freeze all layers in base model
    for param in model.parameters():
        param.requires_grad = False

    # 5. Change classifier head with random seed for reproducibility
    torch.manual_seed(seed)
    model.classifier = nn.Sequential(
        nn.Dropout(p=0.3, inplace=True),
        nn.Linear(in_features=1408, out_features=num_classes),
    )

    return model, transforms
```

```
effnetb2, effnetb2_transforms = create_effnetb2_model()
# effnetb2
effnetb2_transforms
```

```
ImageClassification(
  crop_size=[288]
  resize_size=[288]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BICUBIC
)
```

```
train_dir, test_dir
```

```
(PosixPath('data/pizza_steak_sushi_20_percent/train'),
 PosixPath('data/pizza_steak_sushi_20_percent/test'))
```

```
# Create dataloaders for EffNetB2
from going_modular.going_modular import data_setup
```

```
BATCH_SIZE = 32
train_dataloader_effnetb2, test_dataloader_effnetb2, class_names = data_setup.create_dataloaders(train_dir=train_dir,
                                                                                               test_dir=test_dir,
                                                                                               transform=effnetb2_
                                                                                               batch_size=BATCH_SI

len(train_dataloader_effnetb2), len(test_dataloader_effnetb2), class_names
```

```
(15, 5, ['pizza', 'steak', 'sushi'])
```

```
# Train EffNetB2 feature extractor
from going_modular.going_modular import engine

optimizer = torch.optim.Adam(params=effnetb2.parameters(), lr=1e-3)

loss_fn = torch.nn.CrossEntropyLoss()

set_seeds()
effnetb2_results = engine.train(model=effnetb2,
                                train_dataloader=train_dataloader_effnetb2,
                                test_dataloader=test_dataloader_effnetb2,
                                epochs=10,
                                optimizer=optimizer,
                                loss_fn=loss_fn,
                                device=device)
```

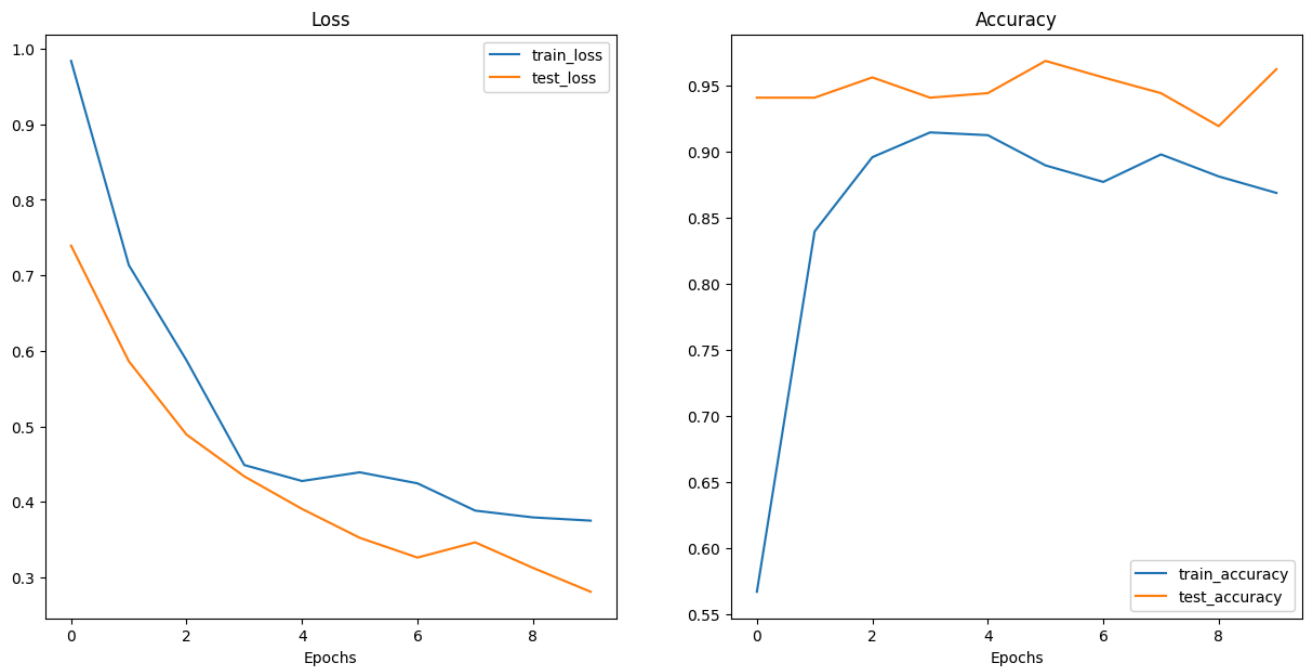
```
100% 10/10 [00:58<00:00, 5.27s/it]
Epoch: 1 | train_loss: 0.9839 | train_acc: 0.5667 | test_loss: 0.7393 | test_acc: 0.9409
Epoch: 2 | train_loss: 0.7135 | train_acc: 0.8396 | test_loss: 0.5862 | test_acc: 0.9409
Epoch: 3 | train_loss: 0.5874 | train_acc: 0.8958 | test_loss: 0.4891 | test_acc: 0.9563
Epoch: 4 | train_loss: 0.4488 | train_acc: 0.9146 | test_loss: 0.4338 | test_acc: 0.9409
Epoch: 5 | train_loss: 0.4277 | train_acc: 0.9125 | test_loss: 0.3907 | test_acc: 0.9443
Epoch: 6 | train_loss: 0.4392 | train_acc: 0.8896 | test_loss: 0.3525 | test_acc: 0.9688
Epoch: 7 | train_loss: 0.4246 | train_acc: 0.8771 | test_loss: 0.3263 | test_acc: 0.9563
Epoch: 8 | train_loss: 0.3885 | train_acc: 0.8979 | test_loss: 0.3465 | test_acc: 0.9443
Epoch: 9 | train_loss: 0.3795 | train_acc: 0.8812 | test_loss: 0.3127 | test_acc: 0.9193
Epoch: 10 | train_loss: 0.3752 | train_acc: 0.8688 | test_loss: 0.2811 | test_acc: 0.9625
```

```
# With label_smoothing=0.1
# Epoch: 1 | train_loss: 1.0005 | train_acc: 0.5708 | test_loss: 0.7872 | test_acc: 0.9347
# Epoch: 2 | train_loss: 0.7704 | train_acc: 0.8500 | test_loss: 0.6603 | test_acc: 0.9409
# Epoch: 3 | train_loss: 0.6679 | train_acc: 0.8896 | test_loss: 0.5883 | test_acc: 0.9500
# Epoch: 4 | train_loss: 0.5608 | train_acc: 0.9146 | test_loss: 0.5522 | test_acc: 0.9318
# Epoch: 5 | train_loss: 0.5528 | train_acc: 0.9125 | test_loss: 0.5239 | test_acc: 0.9352
# Epoch: 6 | train_loss: 0.5718 | train_acc: 0.8875 | test_loss: 0.4973 | test_acc: 0.9597
# Epoch: 7 | train_loss: 0.5609 | train_acc: 0.8854 | test_loss: 0.4864 | test_acc: 0.9472
# Epoch: 8 | train_loss: 0.5457 | train_acc: 0.8958 | test_loss: 0.5050 | test_acc: 0.9318
# Epoch: 9 | train_loss: 0.5338 | train_acc: 0.8896 | test_loss: 0.4809 | test_acc: 0.9193
# Epoch: 10 | train_loss: 0.5417 | train_acc: 0.8500 | test_loss: 0.4654 | test_acc: 0.9625

# Without label_smoothing=0.1
# Epoch: 1 | train_loss: 0.9856 | train_acc: 0.5604 | test_loss: 0.7408 | test_acc: 0.9347
# Epoch: 2 | train_loss: 0.7175 | train_acc: 0.8438 | test_loss: 0.5869 | test_acc: 0.9409
# Epoch: 3 | train_loss: 0.5876 | train_acc: 0.8917 | test_loss: 0.4909 | test_acc: 0.9500
# Epoch: 4 | train_loss: 0.4474 | train_acc: 0.9062 | test_loss: 0.4355 | test_acc: 0.9409
# Epoch: 5 | train_loss: 0.4290 | train_acc: 0.9104 | test_loss: 0.3915 | test_acc: 0.9443
# Epoch: 6 | train_loss: 0.4381 | train_acc: 0.8896 | test_loss: 0.3512 | test_acc: 0.9688
# Epoch: 7 | train_loss: 0.4245 | train_acc: 0.8771 | test_loss: 0.3268 | test_acc: 0.9563
# Epoch: 8 | train_loss: 0.3897 | train_acc: 0.8958 | test_loss: 0.3457 | test_acc: 0.9381
# Epoch: 9 | train_loss: 0.3749 | train_acc: 0.8812 | test_loss: 0.3129 | test_acc: 0.9131
# Epoch: 10 | train_loss: 0.3757 | train_acc: 0.8604 | test_loss: 0.2813 | test_acc: 0.9688
```

```
from helper_functions import plot_loss_curves

plot_loss_curves(effnetb2_results)
```



✓ Preparing and training ViT feature extractor

```
def create_vit_model(num_classes:int=3,
                    seed:int=42):
    """Creates a ViT-B/16 feature extractor model and transforms.

    Args:
        num_classes (int, optional): number of target classes. Defaults to 3.
        seed (int, optional): random seed value for output layer. Defaults to 42.

    Returns:
        model (torch.nn.Module): ViT-B/16 feature extractor model.
        transforms (torchvision.transforms): ViT-B/16 image transforms.
    """
    # Create ViT_B_16 pretrained weights, transforms and model
    weights = torchvision.models.ViT_B_16_Weights.DEFAULT
    transforms = weights.transforms()
    model = torchvision.models.vit_b_16(weights=weights)

    # Freeze all layers in model
    for param in model.parameters():
        param.requires_grad = False

    # Change classifier head to suit our needs (this will be trainable)
    torch.manual_seed(seed)
    model.heads = nn.Sequential(nn.Linear(in_features=768, # keep this the same as original model
                                         out_features=num_classes)) # update to reflect target number of classes

    return model, transforms
```

```
vit, vit_transforms = create_vit_model()
# vit
vit_transforms
```

```
ImageClassification(
  crop_size=[224]
  resize_size=[256]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BILINEAR
)
```

```
# Create dataloaders for ViT
from going_modular.going_modular import data_setup

BATCH_SIZE = 32
train_dataloader_vit, test_dataloader_vit, class_names = data_setup.create_dataloaders(train_dir=train_dir,
                                                                                       test_dir=test_dir,
                                                                                       transform=vit_trans
                                                                                       batch_size=BATCH_SI

len(train_dataloader_vit), len(test_dataloader_vit), class_names

(15, 5, ['pizza', 'steak', 'sushi'])
```

```
# Train ViT feature extractor
from going_modular.going_modular import engine

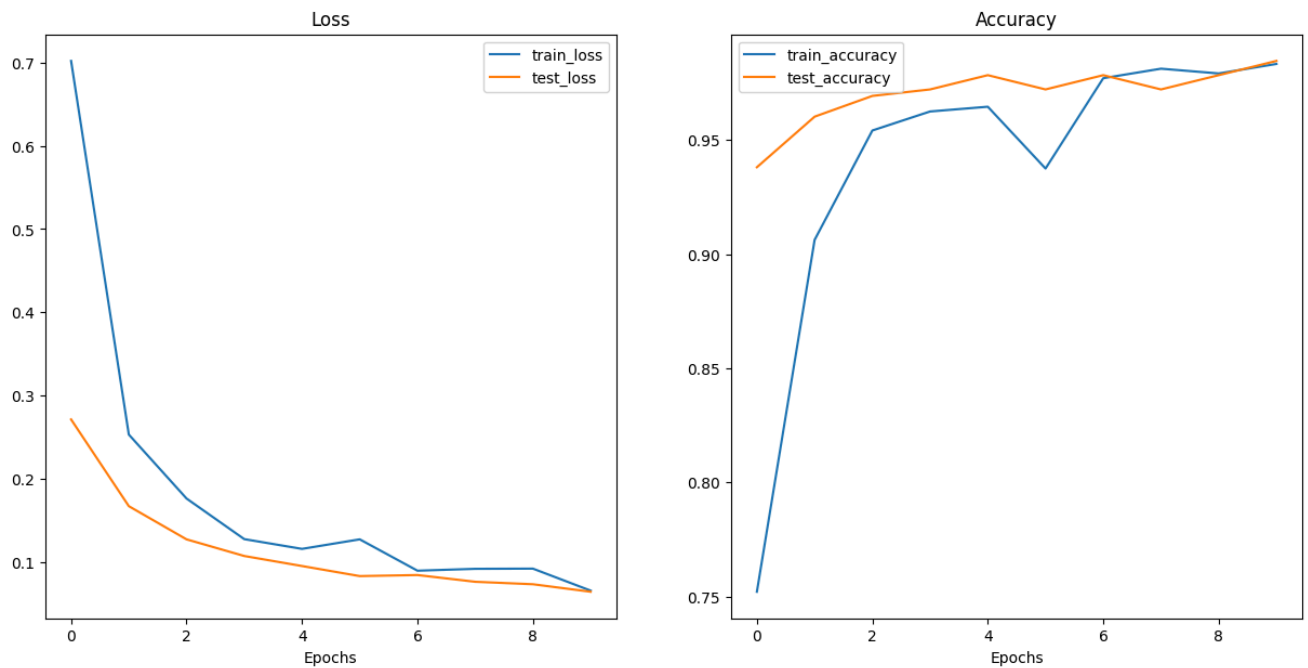
optimizer = torch.optim.Adam(params=vit.parameters(), lr=1e-3)

loss_fn = torch.nn.CrossEntropyLoss()

set_seeds()
vit_results = engine.train(model=vit,
                           train_dataloader=train_dataloader_vit,
                           test_dataloader=test_dataloader_vit,
                           epochs=10,
                           optimizer=optimizer,
                           loss_fn=loss_fn,
                           device=device)
```

```
100% 10/10 [01:25<00:00, 8.49s/it]
Epoch: 1 | train_loss: 0.7020 | train_acc: 0.7521 | test_loss: 0.2714 | test_acc: 0.9381
Epoch: 2 | train_loss: 0.2532 | train_acc: 0.9062 | test_loss: 0.1672 | test_acc: 0.9602
Epoch: 3 | train_loss: 0.1764 | train_acc: 0.9542 | test_loss: 0.1273 | test_acc: 0.9693
Epoch: 4 | train_loss: 0.1276 | train_acc: 0.9625 | test_loss: 0.1074 | test_acc: 0.9722
Epoch: 5 | train_loss: 0.1159 | train_acc: 0.9646 | test_loss: 0.0953 | test_acc: 0.9784
Epoch: 6 | train_loss: 0.1274 | train_acc: 0.9375 | test_loss: 0.0832 | test_acc: 0.9722
Epoch: 7 | train_loss: 0.0897 | train_acc: 0.9771 | test_loss: 0.0845 | test_acc: 0.9784
Epoch: 8 | train_loss: 0.0919 | train_acc: 0.9812 | test_loss: 0.0764 | test_acc: 0.9722
Epoch: 9 | train_loss: 0.0922 | train_acc: 0.9792 | test_loss: 0.0734 | test_acc: 0.9784
Epoch: 10 | train_loss: 0.0658 | train_acc: 0.9833 | test_loss: 0.0644 | test_acc: 0.9847
```

```
plot_loss_curves(vit_results)
```



Get all the images from the test path

Want to make predictions acrosss the test dataset images and time them on GPU to see if they're faster on GPU or CPU...

```
from pathlib import Path
test_image_paths = list(Path(test_dir).glob("*/*.jpg"))
len(test_image_paths)
```

150

```
test_image_paths[:5]
```

```
[PosixPath('data/pizza_steak_sushi_20_percent/test/steak/3553838.jpg'),
 PosixPath('data/pizza_steak_sushi_20_percent/test/steak/2756200.jpg'),
 PosixPath('data/pizza_steak_sushi_20_percent/test/steak/3873655.jpg'),
 PosixPath('data/pizza_steak_sushi_20_percent/test/steak/3100563.jpg'),
 PosixPath('data/pizza_steak_sushi_20_percent/test/steak/595836.jpg')]
```

Get function for making predictions and timing them

See the source here: https://www.learnpytorch.io/09_pytorch_model_deployment/#51-creating-a-function-to-make-predictions-across-the-test-dataset

```
class_names
```

```
['pizza', 'steak', 'sushi']
```

```
"cuda" if torch.cuda.is_available() else "cpu"
```

```
'cuda'
```

```
import pathlib
import torch

from PIL import Image
from timeit import default_timer as timer
from tqdm.auto import tqdm
```



```

from typing import List, Dict

# 1. Create a function to return a list of dictionaries with sample, truth label, prediction, prediction probability
def pred_and_store(paths: List[pathlib.Path],
                  model: torch.nn.Module,
                  transform: torchvision.transforms,
                  class_names: List[str],
                  device: str = "cuda" if torch.cuda.is_available() else "cpu") -> List[Dict]:

    # 2. Create an empty list to store prediction dictionaries
    pred_list = []

    # 3. Loop through target paths
    for path in tqdm(paths):

        # 4. Create empty dictionary to store prediction information for each sample
        pred_dict = {}

        # 5. Get the sample path and ground truth class name
        pred_dict["image_path"] = path
        class_name = path.parent.stem
        pred_dict["class_name"] = class_name

        # 6. Start the prediction timer
        start_time = timer()

        # 7. Open image path
        img = Image.open(path)

        # 8. Transform the image, add batch dimension and put image on target device
        transformed_image = transform(img).unsqueeze(0).to(device)

        # 9. Prepare model for inference by sending it to target device and turning on eval() mode
        model = model.to(device)
        model.eval()

        # 10. Get prediction probability, prediction label and prediction class
        with torch.inference_mode():
            pred_logit = model(transformed_image) # perform inference on target sample
            pred_prob = torch.softmax(pred_logit, dim=1) # turn logits into prediction probabilities
            pred_label = torch.argmax(pred_prob, dim=1) # turn prediction probabilities into prediction label
            pred_class = class_names[pred_label.cpu()] # hardcode prediction class to be on CPU

        # 11. Make sure things in the dictionary are on CPU (required for inspecting predictions later on)
        pred_dict["pred_prob"] = round(pred_prob.max().cpu().item(), 4)
        pred_dict["pred_class"] = pred_class

        # 12. End the timer and calculate time per pred
        end_time = timer()
        pred_dict["time_for_pred"] = round(end_time-start_time, 4)

        # 13. Does the pred match the true label?
        pred_dict["correct"] = class_name == pred_class

        # 14. Add the dictionary to the list of preds
        pred_list.append(pred_dict)

    # 15. Return list of prediction dictionaries
    return pred_list

```

effnetb2_transforms

```

ImageClassification(
  crop_size=[288]
  resize_size=[288]
  mean=[0.485, 0.456, 0.406]
  std=[0.229, 0.224, 0.225]
  interpolation=InterpolationMode.BICUBIC
)

```

▼ Make and time predictions on CPU

```

# Make and time predictions on CPU
effnetb2_preds_on_cpu = pred_and_store(paths=test_image_paths,
                                       model=effnetb2,

```

```
transform=effnetb2_transforms,
class_names=class_names,
device="cpu")
```

100%

150/150 [00:16<00:00, 10.34it/s]

```
# Make and time predictions on CPU
vit_preds_on_cpu = pred_and_store(paths=test_image_paths,
                                  model=vit,
                                  transform=vit_transforms,
                                  class_names=class_names,
                                  device="cpu")
```

100%

150/150 [01:01<00:00, 2.65it/s]

▼ Make and time predictions on GPU

```
# Make and time predictions on GPU
effnetb2_preds_on_gpu = pred_and_store(paths=test_image_paths,
                                       model=effnetb2,
                                       transform=effnetb2_transforms,
                                       class_names=class_names,
                                       device="cuda")

vit_preds_on_gpu = pred_and_store(paths=test_image_paths,
                                  model=vit,
                                  transform=vit_transforms,
                                  class_names=class_names,
                                  device="cuda")
```

100%

150/150 [00:03<00:00, 41.85it/s]

100%

150/150 [00:03<00:00, 49.48it/s]

▼ Comparing prediction times on CPU and GPU

```
import pandas as pd

def get_mean_pred_time(input):
    df = pd.DataFrame(input)
    return df.time_for_pred.mean()

effnetb2_cpu_pred_time = get_mean_pred_time(effnetb2_preds_on_cpu)
effnetb2_cpu_pred_time

np.float64(0.10628800000000001)
```

```
effnetb2_gpu_pred_time = get_mean_pred_time(effnetb2_preds_on_gpu)
vit_cpu_pred_time = get_mean_pred_time(vit_preds_on_cpu)
vit_gpu_pred_time = get_mean_pred_time(vit_preds_on_gpu)

pred_times = {"effnetb2_cpu": effnetb2_cpu_pred_time,
              "vit_cpu": vit_cpu_pred_time,
              "effnetb2_gpu": effnetb2_gpu_pred_time,
              "vit_gpu": vit_gpu_pred_time}

pred_times_df = pd.DataFrame(pred_times, index=[0])
pred_times_df
```

	effnetb2_cpu	vit_cpu	effnetb2_gpu	vit_gpu
0	0.106288	0.407343	0.023818	0.022357



It looks like the predictions on the GPU are much faster than the CPU overall.

And it looks like the ViT model is faster than EffNetB2 on the GPU as well.

So potentially if we had access to a GPU in deployment, a ViT model would be better due to having lower latency (prediction time) as well as better performance.

But if we're focused on deploying to CPU, EffNetB2 wins because of good performance + faster inference time.

Exercise 2. The ViT feature extractor seems to have more learning capacity (due to more parameters) than EffNetB2, how does it go on the larger 20% split of the entire Food101 dataset?

- Train a ViT feature extractor on the 20% Food101 dataset for 5 epochs, just like we did with EffNetB2 in section [10. Creating FoodVision Big](#).

Want to download and split whole Food101 dataset into 20% dataset.

E.g. instead of training on all ~100,000 images in Food101, only train and test on ~20,000 (to save time experimenting).

✓ Create ViT feature extractor for Food101

Need to get a ViT model capable of fitting on Food101 data (freeze the base layers and update the output layers to work with 101 classes).

```
# Create another ViT feature model instance
vit_food101_20_percent, vit_transforms = create_vit_model(num_classes=101)

# Print ViT model summary (uncomment for full output)
from torchinfo import summary
summary(vit_food101_20_percent,
        input_size=(1, 3, 224, 224),
        col_names=["input_size", "output_size", "num_params", "trainable"],
        col_width=20,
        row_settings=["var_names"])
```

Layer (type (var_name))	Input Shape	Output Shape	Param #
Trainable			
VisionTransformer (VisionTransformer)	[1, 3, 224, 224]	[1, 101]	768
Partial			
├─Conv2d (conv_proj)	[1, 3, 224, 224]	[1, 768, 14, 14]	(590,592)
False			
├─Encoder (encoder)	[1, 197, 768]	[1, 197, 768]	151,296
False			
├─Dropout (dropout)	[1, 197, 768]	[1, 197, 768]	--
--			
├─Sequential (layers)	[1, 197, 768]	[1, 197, 768]	--
False			
├─EncoderBlock (encoder_layer_0)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_1)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_2)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_3)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_4)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_5)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_6)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_7)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_8)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_9)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_10)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─EncoderBlock (encoder_layer_11)	[1, 197, 768]	[1, 197, 768]	(7,087,872)
False			
├─LayerNorm (ln)	[1, 197, 768]	[1, 197, 768]	(1,536)
False			
├─Sequential (heads)	[1, 768]	[1, 101]	--
True			
├─Linear (0)	[1, 768]	[1, 101]	77,669
True			
Total params: 85,876,325			
Trainable params: 77,669			

```

Non-trainable params: 85,798,656
Total mult-adds (Units.MEGABYTES): 172.54
=====
Input size (MB): 0.60
Forward/backward pass size (MB): 104.09
Params size (MB): 229.50
Estimated Total Size (MB): 334.19
=====

```

✓ Create Food101 data transforms

Because of the large amount of data, going to use data augmentation to (hopefully) prevent overfitting.

See here: https://www.learnpytorch.io/04_pytorch_custom_datasets/#81-how-to-deal-with-overfitting

```

# Create Food101 training data transforms (only perform data augmentation on the training images)
food101_train_transforms = torchvision.transforms.Compose([
    torchvision.transforms.TrivialAugmentWide(),
    vit_transforms,
])

```

```
food101_train_transforms
```

```

Compose(
  TrivialAugmentWide(num_magnitude_bins=31, interpolation=InterpolationMode.NEAREST, fill=None)
  ImageClassification(
    crop_size=[224]
    resize_size=[256]
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    interpolation=InterpolationMode.BILINEAR
  )
)

```

```

from torchvision import datasets

# Setup data directory
from pathlib import Path
data_dir = Path("data")

# Get training data (~750 images x 101 food classes)
train_data = datasets.Food101(root=data_dir, # path to download data to
                              split="train", # dataset split to get
                              transform=food101_train_transforms, # perform data augmentation on training data
                              download=True) # want to download?

# Get testing data (~250 images x 101 food classes)
test_data = datasets.Food101(root=data_dir,
                             split="test",
                             transform=vit_transforms, # perform normal ViT transforms on test data
                             download=True)

```

```
100%|██████████| 5.00G/5.00G [00:53<00:00, 93.6MB/s]
```

```

# Get Food101 class names
food101_class_names = train_data.classes

# View the first 10
food101_class_names[:10]

```

```

['apple_pie',
 'baby_back_ribs',
 'baklava',
 'beef_carpaccio',
 'beef_tartare',
 'beet_salad',
 'beignets',
 'bibimbap',
 'bread_pudding',
 'breakfast_burrito']

```

✓ Create Food101 20% data splits

Want to split whole Food101 dataset into:

- Train set: 20% of whole original Food101 train dataset
- Test set: 20% of whole original Food101 test dataset

```
def split_dataset(dataset:torchvision.datasets, split_size:float=0.2, seed:int=42):
    """Randomly splits a given dataset into two proportions based on split_size and seed.

    Args:
        dataset (torchvision.datasets): A PyTorch Dataset, typically one from torchvision.datasets.
        split_size (float, optional): How much of the dataset should be split?
            E.g. split_size=0.2 means there will be a 20% split and an 80% split. Defaults to 0.2.
        seed (int, optional): Seed for random generator. Defaults to 42.

    Returns:
        tuple: (random_split_1, random_split_2) where random_split_1 is of size split_size*len(dataset) and
            random_split_2 is of size (1-split_size)*len(dataset).
    """
    # Create split lengths based on original dataset length
    length_1 = int(len(dataset) * split_size) # desired length
    length_2 = len(dataset) - length_1 # remaining length

    # Print out info
    print(f"[INFO] Splitting dataset of length {len(dataset)} into splits of size: {length_1} ({int(split_size*100)}%)

    # Create splits with given random seed
    random_split_1, random_split_2 = torch.utils.data.random_split(dataset,
                                                                    lengths=[length_1, length_2],
                                                                    generator=torch.manual_seed(seed)) # set the rand

    return random_split_1, random_split_2
```

```
# Create training 20% split of Food101
train_data_food101_20_percent, _ = split_dataset(dataset=train_data,
                                                  split_size=0.2)

# Create testing 20% split of Food101
test_data_food101_20_percent, _ = split_dataset(dataset=test_data,
                                                  split_size=0.2)

len(train_data_food101_20_percent), len(test_data_food101_20_percent)

[INFO] Splitting dataset of length 75750 into splits of size: 15150 (20%), 60600 (80%)
[INFO] Splitting dataset of length 25250 into splits of size: 5050 (20%), 20200 (80%)
(15150, 5050)
```

✓ Create DataLoaders for Food101 20 percent data

```
# Turn torch Datasets into DataLoaders
from torch.utils.data import DataLoader

BATCH_SIZE = 32
NUM_WORKERS = 2
train_dataloader_food101 = DataLoader(train_data_food101_20_percent,
                                      batch_size=BATCH_SIZE,
                                      shuffle=True,
                                      num_workers=NUM_WORKERS)

test_dataloader_food101 = DataLoader(test_data_food101_20_percent,
                                     batch_size=BATCH_SIZE,
                                     shuffle=False,
                                     num_workers=NUM_WORKERS)

len(train_dataloader_food101), len(test_dataloader_food101)

(474, 158)
```

✓ Train ViT feature extractor on 20% of Food101 data

Note: The cell below may take 15 mins to run on Google Colab (due to ~15,000 training images and ~5000 testing images).

```
# Loss function
loss_fn = torch.nn.CrossEntropyLoss(label_smoothing=0.1)

# Optimizer
```

```
optimizer = torch.optim.Adam(params=vit_food101_20_percent.parameters(),
                             lr=1e-3)

# Train the model
set_seeds()
vit_food101_20_percent_results = engine.train(model=vit_food101_20_percent,
                                              train_dataloader=train_dataloader_food101,
                                              test_dataloader=test_dataloader_food101,
                                              epochs=5,
                                              optimizer=optimizer,
                                              loss_fn=loss_fn,
                                              device=device)
```

```
100% 5/5 [21:56<00:00, 263.08s/it]
Epoch: 1 | train_loss: 2.8532 | train_acc: 0.4279 | test_loss: 2.0665 | test_acc: 0.6257
Epoch: 2 | train_loss: 2.2209 | train_acc: 0.5931 | test_loss: 1.9249 | test_acc: 0.6657
Epoch: 3 | train_loss: 2.0587 | train_acc: 0.6439 | test_loss: 1.8675 | test_acc: 0.6981
Epoch: 4 | train_loss: 1.9557 | train_acc: 0.6810 | test_loss: 1.8514 | test_acc: 0.6989
Epoch: 5 | train_loss: 1.8883 | train_acc: 0.7009 | test_loss: 1.8297 | test_acc: 0.7078
```

Exercise 3. Make predictions across the 20% Food101 test dataset with the ViT feature extractor from exercise 2 and find the "most wrong" predictions

- The predictions will be the ones with the highest prediction probability but with the wrong predicted label.
- Write a sentence or two about why you think the model got these predictions wrong.

```
# Loop through test DataLoader (with batch size 1)
# Make prediction with model
# Store prediction and prediction probability to dictionary
# Append dictionary to list
# Inspect list

# Easy way: set up batch size of 1 of test data loader - from Sali1997s
# Create batch size of 1 (predict on 1 image at a time)
test_dataloader_food101_batch_size_1 = DataLoader(test_data_food101_20_percent,
                                                  batch_size=1,
                                                  shuffle=False,
                                                  num_workers=NUM_WORKERS)

# Prepare model (do this outside the loop)
vit_food101_20_percent = vit_food101_20_percent.to(device)
vit_food101_20_percent.eval()

# Loop through test DataLoader with batch size 1 and make predictions on each image
# store predictions and truth values to a dictionary and then append dictionary to list for inspection later
vit_food101_pred_list = []
for X, y in tqdm(test_dataloader_food101_batch_size_1):
    # Send data to target device
    X, y = X.to(device), y.to(device)

    # Create empty prediction dictionary (each sample gets a dictionary)
    pred_dict = {}

    # Make predictions
    with torch.inference_mode():
        pred_probs = torch.softmax(vit_food101_20_percent(X), dim=1)
        pred_labels = torch.argmax(pred_probs, dim=1)
        pred_dict["pred_prob"] = torch.max(pred_probs).cpu().numpy()
        pred_dict["pred_label"] = pred_labels.cpu().numpy()[0]
        pred_dict["label"] = y.cpu().numpy()[0]

    vit_food101_pred_list.append(pred_dict)
```

```
100% 5050/5050 [01:25<00:00, 62.06it/s]
```

```
food101_class_names[:5]
```

```
['apple_pie', 'baby_back_ribs', 'baklava', 'beef_carpaccio', 'beef_tartare']
```

```
# Create DataFrame with various columns
pred_df_20_percent = pd.DataFrame(vit_food101_pred_list)
```

```
# Create a column for if the prediction is correct
pred_df_20_percent["is_correct"] = pred_df_20_percent["pred_label"] == pred_df_20_percent["label"]

# Create class name columns (e.g. pred_label=72 -> pred_class="pancakes")
pred_df_20_percent["pred_class"] = pred_df_20_percent["pred_label"].apply(lambda x: food101_class_names[x])
pred_df_20_percent["label_class"] = pred_df_20_percent["label"].apply(lambda x: food101_class_names[x])
pred_df_20_percent.head()
```

	pred_prob	pred_label	label	is_correct	pred_class	label_class	
0	0.7962381	72	72	True	pancakes	pancakes	
1	0.6283233	57	57	True	hummus	hummus	
2	0.19211605	80	80	True	pulled_pork_sandwich	pulled_pork_sandwich	
3	0.57428443	51	51	True	guacamole	guacamole	
4	0.8206848	1	1	True	baby_back_ribs	baby_back_ribs	

Next steps: [Generate code with pred_df_20_percent](#) [New interactive sheet](#)

```
# Get top 10 samples that are "most wrong", e.g. highest pred_prob but wrong prediction - why??
pred_df_20_percent[pred_df_20_percent["is_correct"] == False].sort_values("pred_prob", ascending=False)[:10]
```

	pred_prob	pred_label	label	is_correct	pred_class	label_class	
3107	0.9834614	52	32	False	gyoza	dumplings	
1306	0.94412065	71	44	False	paella	fried_rice	
2848	0.9294365	41	8	False	french_onion_soup	bread_pudding	
708	0.90649515	9	12	False	breakfast_burrito	cannoli	
1726	0.8809202	68	43	False	onion_rings	fried_calamari	
3436	0.8750597	45	58	False	frozen_yogurt	ice_cream	
2287	0.8635277	38	28	False	fish_and_chips	croque_madame	
3209	0.85735285	9	36	False	breakfast_burrito	falafel	
1338	0.85360825	15	51	False	ceviche	guacamole	
1872	0.84609723	79	37	False	prime_rib	filet_mignon	

Comparing the "pred_class" to the "label_class" the model is often wrong on samples that are visually similar.

For example, gyoza and dumplings look quite the same.

The same as paella and fried rice.

And onion rings and fried calamari.

The model is getting confused on similar looking classes and thus predictions are in the right "space" but not necessarily correct compared to the ground truth.

Exercise 4. Evaluate the ViT feature extractor across the whole Food101 test dataset rather than just the 20% version, how does it perform?

- Does it beat the original Food101 paper's best result of 56.4% accuracy?

```
# Check length of Food101 test data
len(test_data)
```

25250

```
# Turn Food101 test data into DataLoader
# Easy way: set up batch size of 1 of test data loader - from Sali1997s
# Create batch size of 1 (predict on 1 image at a time)
test_data_loader_food101_all_data_batch_size_1 = DataLoader(test_data,
                                                             batch_size=1,
                                                             shuffle=False,
                                                             num_workers=NUM_WORKERS)
```

```
len(test_dataloader_food101_all_data_batch_size_1)
```

```
25250
```

```
# Prepare model (do this outside the loop)
vit_food101_20_percent = vit_food101_20_percent.to(device)
vit_food101_20_percent.eval()

# Loop through test DataLoader with batch size 1 and make predictions on each image
# store predictions and truth values to a dictionary and then append dictionary to list for inspection later
vit_food101_all_test_data_pred_list = []
for X, y in tqdm(test_dataloader_food101_all_data_batch_size_1):
    # Send data to target device
    X, y = X.to(device), y.to(device)

    # Create empty prediction dictionary (each sample gets a dictionary)
    pred_dict = {}

    # Make predictions
    with torch.inference_mode():
        pred_probs = torch.softmax(vit_food101_20_percent(X), dim=1)
        pred_labels = torch.argmax(pred_probs, dim=1)
        pred_dict["pred_prob"] = torch.max(pred_probs).cpu().numpy()
        pred_dict["pred_label"] = pred_labels.cpu().numpy()[0]
        pred_dict["label"] = y.cpu().numpy()[0]

    vit_food101_all_test_data_pred_list.append(pred_dict)
```

```
100%
```

```
25250/25250 [07:06<00:00, 53.56it/s]
```

```
# Evaluate model on entire test set
food101_all_test_data_df = pd.DataFrame(vit_food101_all_test_data_pred_list)
food101_all_test_data_df["is_correct"] = food101_all_test_data_df["pred_label"] == food101_all_test_data_df["label"]
food101_all_test_data_df.is_correct.value_counts()
```

	count
is_correct	
True	17849
False	7401

```
dtype: int64
```

```
# Calculate accuracy across Food101 all test
print(f"Accuracy of ViT model on all of Food101 test data: {food101_all_test_data_df.is_correct.value_counts()[1] /
Accuracy of ViT model on all of Food101 test data: 0.2931089108910891
/tmp/ipython-input-3109009526.py:2: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a
print(f"Accuracy of ViT model on all of Food101 test data: {food101_all_test_data_df.is_correct.value_counts()[1] /
```

Exercise 5. Head to [Paperswithcode.com](https://paperswithcode.com) and find the current best performing model on the Food101 dataset.

- What model architecture does it use?

These results are valid as of 26 August 2022:

- Looks like EffNetL2 with extra training data performs the best on Food101 - <https://paperswithcode.com/paper/sharpness-aware-minimization-for-efficiently-1> (96.18% accuracy)
- But Bamboo also looks like it performs the best on another benchmark - <https://paperswithcode.com/paper/bamboo-building-mega-scale-vision-dataset>

Exercise 6. Write down 1-3 potential failure points of our deployed FoodVision models and what some potential solutions might be.

- For example, what happens if someone was to upload a photo that wasn't of food to our FoodVision Mini model?

See the deployed FoodVision Mini model here: https://huggingface.co/spaces/mrdbourke/foodvision_mini

Problems:

1. Someone uploads a photo that isn't food...

- Potential solution: train two models, one that classifies images into "food" or "not food" and if an image is found to be "not food" display a message, if it is found to be "food", classify it into a certain class.

2. Someone uploads a photo of food but our model doesn't have the capacity to classify that certain class...

- Potential solutions:
 1. Get more data for unknown classes (upgrade the model).
 2. Ask someone if the prediction is right/wrong, if it's wrong, track it to a database and update later (active learning - [https://en.wikipedia.org/wiki/Active_learning_\(machine_learning\)](https://en.wikipedia.org/wiki/Active_learning_(machine_learning))).

3. The image quality is poor (food in top left of image rather than center)

- Potential solution: add guidelines to the camera app to hint at taking a photo with the subject in the middle